PROJECT REPORT :

AI Music Recommender

Name: Kartik Sanap

Reg No : 25BAI11282

Course: Fundamentals of AI and ML (Bring your own project)

Github repository: https://github.com/karrtikmeow/ai-song-recommender-

1: The Problem Chosen and Why It Matters:

In the era of infinite digital streaming, users frequently face "decision fatigue" when trying to select music. While major platforms offer static, algorithm-generated playlists, they often fail to capture a user's immediate, highly specific emotional state at a given time of day.

This project addresses the friction of music discovery by building a hyper-contextual **AI Music Recommender**. It matters because emotional resonance is the primary reason people listen to music. By allowing users to input their current mood (e.g., Nostalgic, Focused, Romantic) combined with time-based context (Morning vs. Night), the application bypasses the paradox of choice and instantly delivers a tailored listening experience.

2. Approach to Solving the Problem

To solve this, I developed a web-based application utilizing Python and the Streamlit framework. The core logic relies on mapping distinct emotional states to corresponding musical genres and search queries.

The application workflow is as follows:

1. **User Input:** The user selects from 8 distinct mood categories via a clean, centered UI.
2. **Context Processing:** The system evaluates the time of day to subtly adjust the exact vibe of the recommendation.
3. **API Integration:** The backend sends a dynamically formatted query to the YouTube Music API.
4. **Seamless Delivery:** The application parses the API response and automatically embeds a playable video/audio track directly into the UI with an autoplay feature.

3. Key Decisions Made

- **Framework Selection (Streamlit):** I chose Streamlit over traditional web frameworks like Django or Flask because it allows for rapid, data-driven UI prototyping purely in Python. This allowed me to focus heavily on the recommendation logic and API integration rather than writing extensive HTML/CSS boilerplate.
- **YouTube Music API:** I opted for the YouTube Music API to handle the song retrieval. A major factor in this decision was the ability to embed standard YouTube iframes, which allows for a frictionless "One-Click Music Discovery" and autoplay experience without requiring the user to authenticate through complex OAuth flows.
- **Mood-to-Genre Mapping:** Instead of a complex, heavy machine learning model for a simple MVP, I utilized intelligent heuristic mapping. For example, testing the "Romantic" or "Relaxed" moods with queries designed to pull up tracks akin to "Melting" by Kali Uchis or "Softcore" by The Neighbourhood to ensure the aesthetic matched the user's expectation.

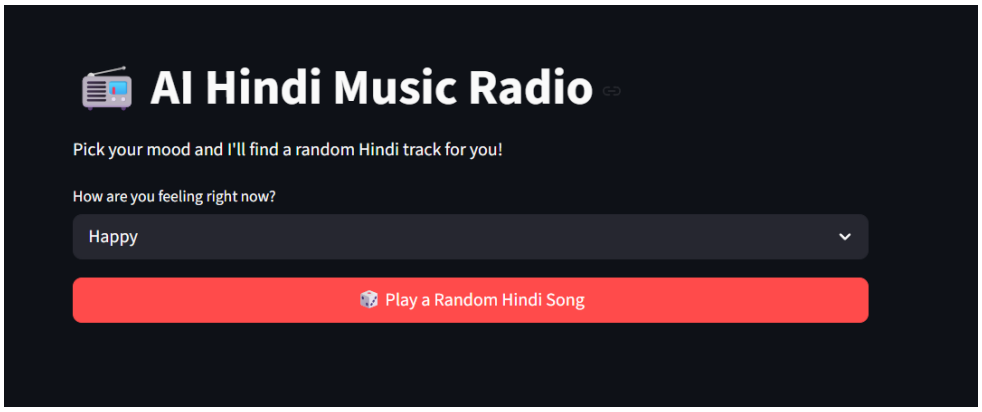
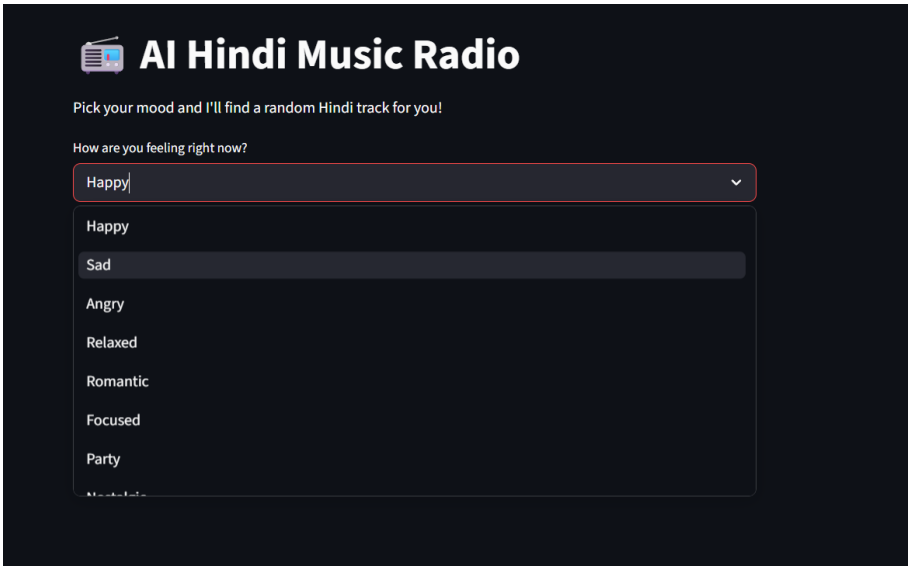
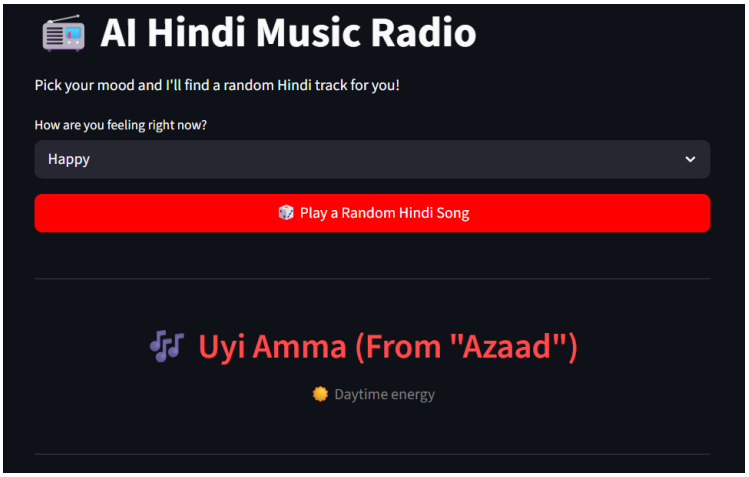
4. Challenges Faced

Developing this application presented several technical hurdles:

- **API Rate Limiting and Data Parsing:** Handling the JSON responses from the YouTube API required careful parsing to extract the correct video IDs and metadata without breaking the app if the API returned an unexpected format.
- **UI State Management:** Ensuring the Streamlit interface updated dynamically to show the embedded player without resetting the entire page state or losing the user's selected mood required managing Streamlit's session state execution flow.
- **Version Control:** Managing the project via Git introduced real-world development challenges, specifically navigating branch merging and remote repository synchronization (such as resolving `fetch first` and remote divergence errors) when pushing the codebase to GitHub.

5. What Was Learned

Building the Mood Radio app provided substantial hands-on experience bridging core computer science concepts with practical application development:

- **API Interactions:** I gained a strong understanding of how to send HTTP requests, handle endpoints, and securely parse external JSON data into a local application.
- **Python Web Development:** I learned how to translate backend Python logic into an interactive, user-facing frontend using Streamlit.
- **Version Control Proficiency:** Troubleshooting Git push rejections and understanding the mechanics of remote tracking branches significantly improved my code management skills.
- **Product Design:** I learned the importance of intuitive UI/UX, utilizing emoji-enhanced visual feedback and a clean layout to make the tool accessible and enjoyable to use.

6 Project screenshots: